

EXPERIMENT 7

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
 - o **public:** available both inside and outside the contract.
 - o **private:** only accessible within the same contract.
 - o **internal:** accessible within the contract and its child contracts.
 - o **external:** can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

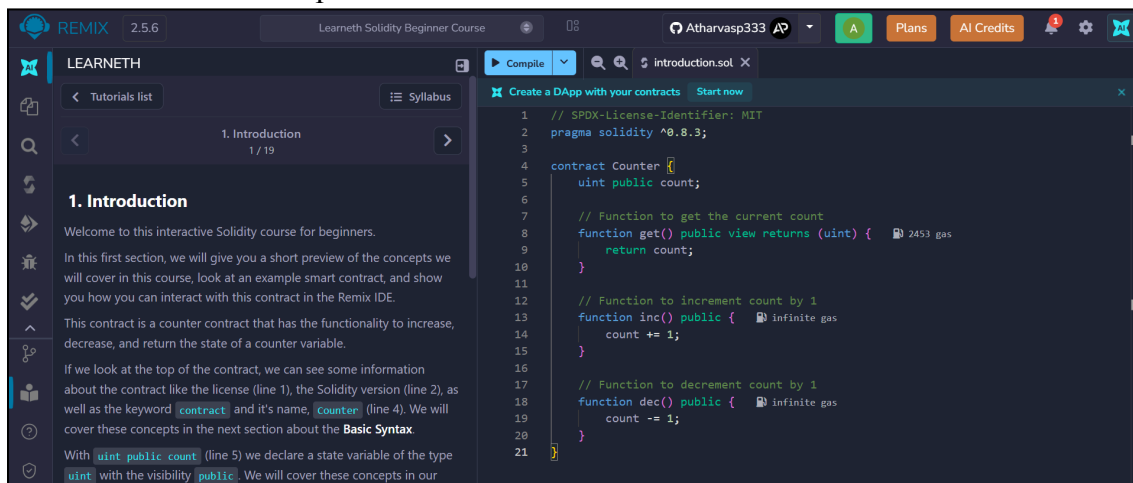
- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

- Tutorial no. 1 – Compile the code



The screenshot displays the Remix IDE interface. On the left, the 'LEARNETH' sidebar shows the '1. Introduction' tutorial, which explains the purpose of the counter contract and its basic syntax. The main editor area on the right contains the Solidity code for the 'Counter' contract. The code includes a license, pragma statement, and three functions: 'get()', 'inc()', and 'dec()'. Gas costs are indicated for the 'get()' and 'dec()' functions.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

- Tutorial no. 1 – Deploy the contract

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is open. Under 'Environment', it shows 'Remix VM' and 'Osaka'. Under 'Deployed Contracts', it lists a 'Counter' contract at address '0xd91...39138' deployed '2min ago'. The main editor displays a Solidity contract named 'Counter' with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

At the bottom, a transaction log shows a successful deployment: '[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x181...6f086'.

- Tutorial no. 1 – get

The screenshot shows the Remix IDE interface. In the 'DEPLOY & RUN TRANSACTIONS' sidebar, the 'Functions' section is expanded, showing a 'Select a function to interact with...' dropdown. Below it, the 'get' function is selected, with a 'uint256: 0' input field and a 'Call' button. The main editor shows the same Solidity contract as before. The transaction log at the bottom displays a message: 'The transaction has been reverted to the initial state. Note: The called function should be payable if you send value and the value you send should be less than your current balance. If the transaction failed for not having enough gas, try increasing the gas limit gently.'

- Tutorial no. 1 – Increment

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. Under 'Environment', 'Remix VM' is selected with 'Osaka' as the network. 'Account 1' (0x5B3...eddC4) has a balance of 99.999 ETH. In the 'Low level interaction' section, the 'inc' transaction is selected. The 'Value' is set to 1 wei, and the 'Gas limit' is set to 'auto' (0). A 'Transact' button is visible. Below this, the 'Transactions recorder' shows 6 transactions. On the right, the 'introduction.sol' file is open, displaying Solidity code for a 'Counter' contract. The code includes a 'get()' function, an 'inc()' function (which is highlighted), and a 'dec()' function. The 'AI copilot' button is visible at the bottom right of the code editor.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

- Tutorial no. 1 – Decrement

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel is active. Under 'Environment', 'Remix VM' is selected with 'Osaka' as the network. 'Account 1' (0x5B3...eddC4) has a balance of 99.999 ETH. In the 'Low level interaction' section, the 'dec' transaction is selected. The 'Value' is set to 1 wei, and the 'Gas limit' is set to 'auto' (0). A 'Transact' button is visible. Below this, the 'Transactions recorder' shows 7 transactions. On the right, the 'introduction.sol' file is open, displaying Solidity code for a 'Counter' contract. The code includes a 'get()' function, an 'inc()' function, and a 'dec()' function (which is highlighted). The 'AI copilot' button is visible at the bottom right of the code editor.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

- Tutorial no. 2

The screenshot shows the Learneth Solidity Beginner Course interface. The left sidebar contains a 'Tutorials list' with '2. Basic Syntax' selected (2/19). Below it, an 'Assignment' section lists four tasks: 1. Delete the HelloWorld contract and its content. 2. Create a new contract named 'MyContract'. 3. The contract should have a public state variable called 'name' of the type string. 4. Assign the value 'Alice' to your new variable. At the bottom of the sidebar, there are 'Check Answer' and 'Show answer' buttons, and a green status bar says 'Well done! No errors.' The main editor area shows a Solidity contract named 'MyContract' with a public state variable 'name' of type 'string' assigned the value 'Alice'. The code is as follows:

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

The top bar shows the user 'Atharvasp333' and various course navigation options like 'Plans' and 'AI Credits'.

- Tutorial no. 3

The screenshot shows the Learneth Solidity Beginner Course interface for Tutorial 3: Primitive Data Types. The left sidebar shows '3. Primitive Data Types' selected (3/19). The 'Assignment' section lists three tasks: 1. Create a new variable 'newAddr' that is a 'public address' and give it a value that is not the same as the available variable 'addr'. 2. Create a 'public' variable called 'neg' that is a negative number, decide upon the type. 3. Create a new variable, 'newU' that has the smallest 'uint' size type and the smallest 'uint' value and is 'public'. A tip below the tasks says: 'Tip: Look at the other address in the contract or search the internet for an Ethereum address.' The main editor area shows a Solidity contract with various primitive data types: 'uint', 'int8', 'int', 'address', 'bool', and 'uint8'. The code is as follows:

```
17 uint public u = 123; // uint is an alias for uint256
18
19 /*
20  Negative numbers are allowed for int types.
21  Like uint, different ranges are available from int8 to int256
22  */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool; // false
32 uint public defaultUInt; // 0
33 int public defaultInt; // 0
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000
35 address public newAddr = 0x0000000000000000000000000000000000000001;
36 int public neg = -10;
37 uint8 public newU = 0;
38 }
```

The top bar shows the user 'Atharvasp333' and various course navigation options like 'Plans' and 'AI Credits'.

• Tutorial no. 4

LEARNETH

Tutorials list

4. Variables
4 / 19

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#). Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ **Assignment**

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Variables.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

[Explain contract](#) AI copilot

• Tutorial no. 5

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable
5 / 19

variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ **Assignment**

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

SimpleStorage.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public { 22536 gas
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) { 2475 gas
16        return num;
17    }
18
19    function get_b() public view returns (bool) { 2539 gas
20        return b;
21    }
22 }
```

[Explain contract](#) AI copilot

• Tutorial no. 6

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

5.2 Functions - View and Pure 6 / 19

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Code Editor:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x = x + y;
19    }
20 }

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

• Tutorial no. 7

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

5.3 Functions - Modifiers and Constructors 7 / 19

Modifiers are especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

- Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Code Editor:

```

13     owner = msg.sender;
14 }
15
16 // Modifier to check that the caller is the owner of
17 // the contract.
18 modifier onlyOwner() {
19     require(msg.sender == owner, "Not owner");
20     // Underscore is a special character only used inside
21     // a function modifier and it tells Solidity to
22     // execute the rest of the code.
23     _;
24 }
25
26 modifier increase(uint _value) {
27     x += _value;
28     _;
29 }
30
31 function increaseX(uint _value) public increase(_value) { infinite gas
32
33 }
34
35 // Modifiers can take inputs. This modifier checks that the
36 // address passed in is not the zero address.
37 modifier validAddress(address _addr) {
38     require(_addr != address(0), "Not valid address");
39     _;

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

- Tutorial no. 8

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333

5.4 Functions - Inputs and Outputs 8 / 19

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

59 {
60     (uint i, bool b, uint j) = returnMany();
61
62     // Values can be left out.
63     (uint x, , uint y) = (4, 5, 6);
64
65     return (i, b, j, x, y);
66 }
67
68 // Cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public { Infinite gas
72
73     // Can use array for output
74     uint[] public arr;
75
76     function arrayOutput() public view returns (uint[] memory) { Infinite gas
77         return arr;
78     }
79
80     function returnTwo() public pure returns (int p, bool q) { 472 gas
81         p = -2;
82         q = true;
83     }
84 }

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

- Tutorial no. 9

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333

6. Visibility 9 / 19

`Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

Assignment

Create a new function in the `child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

51 // State variables cannot be external so this code won't compile.
52 // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     // function testPrivateFunc() public pure returns (string memory) {
59     //     return privateFunc();
60     // }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

- Tutorial no. 10

LEARNETH 7.1 Control Flow - If/Else 10 / 19

eise it

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the `If/Else` statement.

★ **Assignment**

Create a new function called `evencheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evencheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint _x) public pure returns (uint) {
16        // if (_x < 10) {
17        //     return 1;
18        // }
19        // re
20        // shorthand way to write if / else statement
21        return _x < 10 ? 1 : 2;
22    }
23
24    function evenCheck(uint a) public pure returns (bool) {
25        return a % 2 == 0 ? true : false;
26    }
27
28 }
  
```

[Explain contract](#) AI copilot

- Tutorial no. 11

LEARNETH 7.2 Control Flow - Loops 11 / 19

the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the `for` loop to be terminated after the sixth iteration.

Watch a video tutorial on `Loop` statements.

★ **Assignment**

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the `for` loop, increment the `count` variable by 1.
- Try to get the `count` variable to be equal to 9, but make sure you don't edit the `break` statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

4 contract Loop {
5     uint public count;
6
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10            if (i == 5) {
11                // Skip to next iteration with continue
12                continue;
13            }
14            if (i == 5) {
15                // Exit loop with break
16                break;
17            }
18            count++;
19        }
20
21        // while loop
22        uint j;
23        while (j < 10) {
24            j++;
25        }
26    }
27
28 }
  
```

[Explain contract](#) AI copilot

- Tutorial no. 12

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 8.1 Data Structures - Arrays 12 / 19

Operator on other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     uint[3] public arr3 = [0, 1, 2];
9     // Fixed sized array, all elements initialize to 0
10    uint[10] public myFixedSizeArr;
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) {
20        return arr3;
21    }
22
23    function push(uint i) public {
24        // Append to array
25        // This will increase the array length by 1.
26        arr.push(i);
27    }

```

AI copilot

- Tutorial no. 13

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 8.2 Data Structures - Mappings 13 / 19

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         return balances[_addr];
10    }
11
12    function set(address _addr) public {
13        balances[_addr] = _addr.balance;
14    }
15
16    function remove(address _addr) public {
17        delete balances[_addr];
18    }
19 }
20
21 contract NestedMapping {
22     // Nested mapping (mapping from address to another mapping)
23     mapping(address => mapping(uint => bool)) public nested;
24
25     function get(address _addr1, uint _i) public view returns (bool) {
26         // You can get values from a nested mapping
27         // even when it is not initialized

```

AI copilot

- Tutorial no. 14

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 8.3 Data Structures - Structs 14 / 19

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs
To access a member of a struct we can use the dot operator (line 33).

Updating structs
To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).
[Watch a video tutorial on Structs.](#)

★ **Assignment**
Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
30 // you don't actually need this function.
31 function get(uint _index) public view returns (string memory text, bool comple
32     Todo storage todo = todos[_index];
33     return (todo.text, todo.completed);
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public { infinite gas
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public { 28995 gas
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47
48 function remove(uint _index) public { infinite gas
49     delete todos[_index];
50 }
51 }
```

- Tutorial no. 15

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 8.4 Data Structures - Enums 15 / 19

representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value
We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ **Assignment**
1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
25 // Rejected - 3
26 // Canceled - 4
27 function get() public view returns (Status) { 2656 gas
28     return status;
29 }
30
31 // Update status by passing uint into input
32 function set(Status _status) public { undefined gas
33     status = _status;
34 }
35
36 // You can update to a specific enum like this
37 function cancel() public { 24545 gas
38     status = Status.Canceled;
39 }
40
41 // delete resets the enum to its first value, 0
42 function reset() public { 24434 gas
43     delete status;
44 }
45
46 function getSize() public view returns (Size) { 2582 gas
47     return sizes;
48 }
49 }
```

- Tutorial no. 16

LEARNETH

Tutorials list

9. Data Locations 16 / 19

of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) myStructs;
11
12    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory)
13        _f(arr, map, myStructs[1]);
14
15    MyStruct storage myStruct = myStructs[1];
16    myStruct.foo = 4;
17
18    MyStruct memory myMemStruct = MyStruct(0);
19    MyStruct memory myMemStruct2 = myStruct;
20    myMemStruct2.foo = 1;
21
22    MyStruct memory myMemStruct3 = myStruct;
23    myMemStruct3.foo = 3;
24
25    return (myStruct, myMemStruct2, myMemStruct3);
26
27

```

Explain contract AI copilot

- Tutorial no. 17

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei 17 / 19

Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     uint public oneGwei = 1 gwei;
7     bool public isOneGwei = (1 gwei == 10**9);
8
9     // 1 wei is equal to 1
10    bool public isOneWei = 1 wei == 1;
11
12    uint public oneEther = 1 ether;
13    // 1 ether is equal to 10^18 wei
14    bool public isOneEther = 1 ether == 1e18;
15
16

```

Explain contract AI copilot

• Tutorial no. 18

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 10.2 Transactions - Gas and Gas Price 18 / 19

gas that they are willing to pay for. If they set the limit too low, their transaction can run out of *gas* before being completed, reverting any changes being made. In this case, the *gas* was consumed and can't be refunded.

Learn more about *gas* on ethereum.org.

[Watch a video tutorial on Gas and Gas Price.](#)

★ **Assignment**

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Contract Code:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

• Tutorial no. 19

LEARNETH 2.5.6 Learneth Solidity Beginner Course Atharvasp333 Plans AI Credits

Tutorials list 10.3 Transactions - Sending Ether 19 / 19

[Watch a video tutorial on Sending Ether.](#)

★ **Assignment**

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Contract Code:

```

50 }
51 }
52
53 contract Charity {
54     address public owner;
55
56     constructor() {
57         // 164845 gas 140400 gas
58         owner = msg.sender;
59     }
60
61     function donate() public payable {} // 141 gas
62
63     function withdraw() public {
64         // infinite gas
65         uint amount = address(this).balance;
66         (bool sent, ) = owner.call{value: amount}("");
67         require(sent, "Failed to send Ether");
68     }
69 }

```

[Create a DApp with your contracts](#) [Start now](#)

[Explain contract](#) AI copilot

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.